

PRACTICAL WEEK 2

Task 1: Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Code:

```
UM PID0 6.09 File: 4.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[])
{
    int source, destination;
    ssize_t bytesRead, bytesWritten;
    char buffer[BUFFER_SIZE];

    // Check command-line arguments
    if (argc != 3)
    {
        printf("Usage: %s <source_file> <destination_file>\n", argv[0]);
        return 1;
    }

    // Open source file in read-only mode
    source = open(argv[1], O_RDONLY);

    if (source == -1)
    {
        perror("Error opening source file");
        return 1;
    }

    // Open/create destination file
    destination = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC,
                       0644);

    if (destination == -1)
    {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    // Read from source and write to destination
    while ((bytesRead = read(source, buffer, BUFFER_SIZE)) > 0)
    {
        bytesWritten = write(destination, buffer, bytesRead);

        if (bytesWritten == -1)
        {
            perror("Error writing to destination file");
            close(source);
            close(destination);
            return 1;
        }
    }

    if (bytesRead == -1)
    {
        perror("Error reading source file");
        close(source);
    }
}
```

Output:

```
Last login: Wed Aug 26 12:14:28 on tty000
nrk@nallas-mac-Book-Air ~ % nano 4.c

nrk@nallas-mac-Book-Air ~ % gcc 4.c -o 4
nrk@nallas-mac-Book-Air ~ % ./4
Usage: ./4 <source_file> <destination_file>
nrk@nallas-mac-Book-Air ~ %
```

Task-2: Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

Code:

```
nm PICO 5.09 File: 5.c Modified
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

#define BUFFER_SIZE 1024

int main(int argc, char *argv[])
{
    int fd;
    char buffer[BUFFER_SIZE];
    ssize_t bytesRead, bytesWritten;

    // Check whether filename is provided
    if (argc != 2)
    {
        printf("Usage: %s <filename>\n", argv[0]);
        return 1;
    }

    // Open the file
    fd = open(argv[1], O_RDONLY);

    if (fd == -1)
    {
        perror("Error opening file");
        return 1;
    }

    // Read file and display contents
    while ((bytesRead = read(fd, buffer, BUFFER_SIZE)) > 0)
    {
        bytesWritten = write(STDOUT_FILENO, buffer, bytesRead);

        if (bytesWritten == -1)
        {
            perror("Error writing output");
            close(fd);
            return 1;
        }
    }

    // Check for read error
    if (bytesRead == -1)
    {
        perror("Error reading file");
        close(fd);
        return 1;
    }

    // Close the file
    close(fd);

    return 0;
}
```

Output:

```
nrk@nallas-mac-Book-Air ~ % nano 5.c
nrk@nallas-mac-Book-Air ~ % gcc 5.c -o 5
nrk@nallas-mac-Book-Air ~ % ./5
Usage: ./5 <filename>
nrk@nallas-mac-Book-Air ~ %
```

System Call	Purpose	Kernel Service
execve()	Starts the cat program	Process/program execution
openat()	Opens sample.txt	File-system access
read()	Reads file contents	File I/O
write()	Displays contents on terminal	Output/I/O
close()	Closes the file descriptor	Resource management
exit_group()	Terminates the process	Process termination